



UNIVERSIDAD DE MONTERREY

Práctica 1

Microprocesadores

M.C. Alejandro Omar Reyes Guía

Nombre: Eugenio Romo García
Nombre: Ricardo Garza Benítez

Mat: 612348
Mat: 645972

"Damos nuestra palabra de que hemos hecho esta actividad con integridad académica"

Monterrey, N. L., a 3 de Junio del 2026

Introducción

El microcontrolador PIC16F887, desarrollado por Microchip Technology, es un dispositivo de arquitectura RISC de 8 bits ampliamente empleado en aplicaciones de control embebido y automatización. Su diseño integra en un solo encapsulado una unidad central de procesamiento (CPU), memoria de programa tipo Flash, memoria de datos SRAM, memoria EEPROM no volátil y múltiples periféricos configurables, entre los que destacan cinco puertos de entrada/salida de propósito general. Esta integración lo convierte en una plataforma ideal para introducirse al desarrollo con microcontroladores, ya que permite explorar desde el manejo básico de puertos digitales hasta la implementación de interfaces más complejas sobre una misma arquitectura. Esta primera práctica tiene como propósito establecer las bases operativas del flujo de trabajo que se utilizará a lo largo del curso.

El entorno de desarrollo utilizado es MPLAB X IDE en conjunto con el compilador XC8, herramientas de Microchip que permiten escribir, compilar y depurar código en lenguaje C para toda la familia de microcontroladores PIC. La configuración inicial del microcontrolador, mediante los bits de configuración o fuses, es un paso crítico que determina aspectos fundamentales como la fuente de reloj, la habilitación del Watchdog Timer y la protección de la memoria de programa. En esta práctica se empleó un oscilador de cristal externo de tipo XT a 4 MHz, lo cual establece la frecuencia base sobre la cual se calculan los retardos y tiempos de respuesta del sistema.

Las dos actividades planteadas en esta práctica abordan el manejo del puerto D (PORTD) del PIC16F887 mediante dos programas distintos: un contador binario de 6 bits y una secuencia tipo caminata de 8 bits. Ambos programas constituyen ejercicios canónicos en la iniciación al uso de microcontroladores, ya que permiten comprender la escritura directa sobre registros de puerto, el uso de operadores a nivel de bit en C, el manejo de ciclos iterativos y la introducción de retardos por software. La visualización de los resultados se realizó mediante arreglos de LEDs conectados a las salidas del puerto, lo que proporcionó una verificación inmediata y directa del comportamiento del código ejecutado en el dispositivo físico.

Marco Teórico

PIC16F887 y Arquitectura RISC

El PIC16F887 es un microcontrolador de 8 bits perteneciente a la familia PIC16F de Microchip Technology, basado en arquitectura Harvard modificada con un conjunto reducido de instrucciones (RISC). Su CPU opera sobre un bus de datos de 8 bits y un bus de programa de 14 bits, ejecutando la mayoría de las instrucciones en un solo ciclo de instrucción equivalente a cuatro ciclos de reloj. El dispositivo cuenta con 8 KB de memoria Flash para el programa de usuario, 368 bytes de SRAM y 256 bytes de EEPROM de datos. Incorpora cinco puertos de entrada/salida bidireccionales (PORTA, PORTB, PORTC, PORTD y PORTE) configurables mediante registros TRIS, donde un bit en cero define un pin como salida y un bit en uno lo define como entrada. En esta práctica se utilizó el PORTD configurado completamente como salida mediante la instrucción TRISD = 0b00000000, lo que habilita sus ocho pines para manejar los LEDs indicadores [1].

Bits de Configuración (Fuses)

Los bits de configuración, comúnmente denominados fuses, son un conjunto de bits almacenados en una región especial de la memoria de programa del PIC16F887 que determinan el comportamiento del microcontrolador a nivel de hardware desde el momento en que inicia su ejecución. Su configuración se realiza en el código fuente mediante directivas `#pragma config` y queda grabada en el dispositivo durante la programación. En esta práctica se estableció `FOSC = XT` para seleccionar el oscilador de cristal externo de 4 MHz como fuente de reloj, `WDTE = OFF` para deshabilitar el Watchdog Timer durante las pruebas, `PWRTTE = OFF` para omitir el temporizador de encendido, y `LVP = OFF` para deshabilitar la programación en bajo voltaje. La correcta configuración de estos bits es indispensable para que el microcontrolador opere con los parámetros de tiempo y voltaje para los que se diseñó el circuito [1].

MPLAB X IDE y Compilador XC8

MPLAB X IDE es el entorno de desarrollo integrado oficial de Microchip Technology para el diseño de firmware en microcontroladores y microprocesadores PIC y AVR. Soporta múltiples compiladores, siendo XC8 el compilador de lenguaje C optimizado para la familia PIC de 8 bits. El flujo de trabajo dentro de MPLAB X comprende la creación del proyecto, la configuración del dispositivo objetivo, la escritura del código fuente en C, la compilación y enlazado para generar el archivo `.hex`, y finalmente la programación del dispositivo mediante herramientas como el PICkit o el ICD. El compilador XC8 traduce el código de alto nivel a instrucciones en lenguaje ensamblador específicas para la arquitectura del PIC16F887, respetando las restricciones de memoria y los tiempos de ciclo definidos por la frecuencia del oscilador. La función `__delay_ms()`, disponible como macro del compilador, genera retardos calibrados en función de la definición `_XTAL_FREQ` que refleja la frecuencia del cristal utilizado [2].

Actividad 1: Contador Binario de 6 bits

Un contador binario es una secuencia lógica en la que un valor numérico entero se incrementa de forma cíclica desde cero hasta un límite definido, representándose cada valor en notación binaria sobre los bits del puerto de salida. En el PIC16F887, el PORTD de 8 bits puede representar valores de 0 a 255; sin embargo, en esta actividad se restringe la cuenta a 6 bits mediante la aplicación de una máscara lógica AND con el valor `0b00111111`. Esta operación garantiza que únicamente los seis bits menos significativos (D0–D5) se activen durante la secuencia, dejando los pines D6 y D7 en nivel bajo de forma permanente independientemente del valor del contador. La variable contador de tipo `uint8_t` se incrementa en cada iteración del bucle `while(1)` y se reinicia a cero al alcanzar el valor 64, completando así un ciclo de 64 estados que corresponden exactamente a los $2^6 = 64$ combinaciones posibles de 6 bits. El retardo de 200 ms entre transiciones hace que la secuencia sea perceptible visualmente en los LEDs conectados al puerto [1].

Actividad 2: Secuencia Tipo Caminata (Walking Bit) de 8 bits

Una secuencia tipo caminata o walking bit es un patrón de activación en el que un único bit en alto se desplaza de forma secuencial a través de todos los bits de un registro, produciendo la impresión visual de un punto de luz que recorre los LEDs de un extremo al otro y regresa. En C, este desplazamiento se implementa mediante el operador de corrimiento a la izquierda (`<<`), donde la expresión `(1 << i)` genera un valor con exactamente un bit activo en la posición `i`. El primer bucle `for` recorre los índices de 0 a 7 desplazando el bit hacia la izquierda (D0 → D7),

mientras que el segundo for lo regresa de la posición 6 a la 1, evitando repetir los extremos y logrando un movimiento de ida y vuelta continuo. Cada estado permanece visible durante 150 ms antes de la siguiente transición. Esta técnica de manipulación directa de bits mediante operadores de corrimiento es un recurso fundamental en el control de hardware a bajo nivel, ya que permite activar o desactivar pines individuales sin afectar el estado de los demás [1].

Simulación

[Simulación — Imagen de Proteus, Actividad 1: Contador Binario]

[Simulación — Imagen de Proteus, Actividad 2: Walking Bit]

Foto del circuito en Físico

[Foto del circuito físico armado en protoboard]

Resultados

Actividad 1 — Contador Binario de 6 bits

Durante la ejecución del primer programa, se pudo observar que los seis LEDs conectados a los pines RD0–RD5 del PORTD realizaron una cuenta binaria cíclica de forma correcta. A partir del estado inicial con todos los LEDs apagados (valor 0), el patrón de activación avanzó de manera ordenada incrementando el valor binario en una unidad cada 200 ms, hasta alcanzar el estado 0b00111111 correspondiente al valor decimal 63, momento en que el contador reiniciaba automáticamente a cero. Los pines RD6 y RD7, cubiertos por la máscara AND 0b00111111, permanecieron en nivel bajo durante toda la ejecución, confirmando que la operación de enmascaramiento funcionó conforme a lo esperado. El proceso de compilación en MPLAB X con el compilador XC8 y la transferencia del archivo .hex al PIC16F887 se completaron sin errores, lo que permitió validar tanto el entorno de desarrollo como la configuración de los fuses establecida en el código.

La cuenta cíclica de 64 estados (0 a 63) se ejecutó de forma continua e indefinida gracias al bucle while(1), y el periodo de cada ciclo completo resultó en un tiempo aproximado de 12.8 segundos (64 estados × 200 ms por estado), lo cual fue perceptible visualmente y permitió verificar el comportamiento sin necesidad de instrumentación adicional. La inicialización TRISD = 0b00000000 configuró correctamente todos los pines del PORTD como salidas digitales, y la inicialización PORTD = 0 aseguró un estado limpio de partida al encender el microcontrolador.

Actividad 2 — Secuencia Walking Bit de 8 bits

En el segundo programa, los ocho LEDs conectados al PORTD reprodujeron con precisión la secuencia de caminata programada. El bit activo inició en RD0 y avanzó hacia RD7 mediante el operador de corrimiento a la izquierda, encendiendo un solo LED a la vez durante 150 ms antes de desplazarse al siguiente. Al alcanzar RD7, el segundo bucle for invirtió la dirección de desplazamiento regresando desde RD6 hasta RD1, completando el recorrido de vuelta. Esta secuencia bidireccional se repitió de forma continua e indefinida dentro del bucle while(1) externo. El comportamiento visual fue el esperado: en ningún momento se observaron dos LEDs activos simultáneamente, lo que confirmó que el operador $(1 \ll i)$ genera efectivamente un único bit activo por iteración.

La carga del archivo .hex al microcontrolador se realizó sin inconvenientes, reafirmando la correcta configuración del oscilador XT a 4 MHz y los fuses compartidos con la actividad anterior. El retardo de 150 ms por estado produjo un efecto visual fluido y distinguible a simple vista. En conjunto, ambas actividades permitieron verificar de manera exitosa el flujo completo de desarrollo: escritura del código en C, configuración de fuses, compilación con XC8, programación del dispositivo y observación del comportamiento en hardware real, sentando las bases del método de trabajo que se empleará durante el resto del curso.

Conclusiones Individuales

Ricardo Garza Benítez

Esta práctica me permitió poner en marcha de forma completa el flujo de trabajo que usaremos durante el semestre, y hacerlo en hardware real le dio un peso distinto a lo que hasta ahora había sido solamente teoría. Lo que más me llamó la atención fue la relevancia de los bits de configuración: antes de escribir una sola línea de lógica, es necesario decirle al microcontrolador qué oscilador usar, si el Watchdog está activo o no, y cómo se maneja la programación. Un fuse mal configurado puede hacer que todo lo demás falle sin un mensaje de error evidente, y eso exige un nivel de atención al detalle que no siempre está presente cuando se trabaja en simulación. Al armar el circuito físico con los LEDs y el PIC16F887, comprobé que la asignación TRISD = 0b00000000 y la escritura directa sobre PORTD producen exactamente los patrones que el código especifica, sin capas intermedias que abstraigan el comportamiento. Ver el contador binario avanzar en los LEDs y luego el walking bit recorriendo los pines de forma secuencial fue una validación directa de que tanto el código como el circuito estaban correctos. Creo que entender este nivel de control directo sobre el hardware es lo que distingue el desarrollo con microcontroladores de otras formas de programación, y es algo que voy a necesitar dominar con solidez para los proyectos que vienen.

Eugenio Romo García

Esta primera práctica representó un punto de partida importante para comprender la diferencia entre escribir código para un sistema operativo y programar un microcontrolador directamente. A diferencia de entornos de alto nivel, aquí cada instrucción tiene un efecto concreto y medible sobre un pin físico: configurar TRISD como salida y escribir un valor en PORTD es suficiente para encender o apagar un LED, sin abstracciones intermedias. Lo que más me ayudó a clarificar conceptos fue el uso del operador de corrimiento en la actividad del walking bit, ya que ilustra de manera muy directa cómo los operadores a nivel de bit permiten manipular posiciones individuales dentro de un registro de 8 bits. Más allá de los circuitos específicos, considero que el valor central de esta práctica estuvo en aprender el proceso: crear el proyecto en MPLAB X, configurar los fuses, compilar con XC8 y transferir el .hex al dispositivo. Ese flujo es la base sobre la que se construirán todas las prácticas siguientes.

Referencias

[1] Microchip Technology. (2007). *PIC16F882/883/884/886/887 Data Sheet*. Microchip Technology Inc. <https://ww1.microchip.com/downloads/en/DeviceDoc/41291D.pdf>

[2] Microchip Technology. (2023). *MPLAB XC8 C Compiler User's Guide*. Microchip Technology Inc. <https://ww1.microchip.com/downloads/en/DeviceDoc/50002053G.pdf>